COMP512 P1

Karl Wehbe, Ion Basha

May 22, 2026

1 Introduction

In this project, we implemented a simple distributed system consisting of a Middleware server and three resource managers ('Flights', 'Cars', 'Rooms'). This report details the architecture used to implement this system, using both Remote Method Invocation (RMI) and Transmission Control Protocol (TCP) for communication between processes.

2 Architecture

2.1 RMI

We first implemented the distributed system described in the Introduction using RMI for inter-process communication. Each Resource Manager (Flights, Cars, Rooms) runs an RMI registry on its host, where it registers a remote object. The Middleware obtains proxies of these remote objects from the registries, creates its own remote object, and binds it to its local registry. This architecture forms a client–server relationship between the Middleware (in this context, the client) and the Resource Managers (in this context, the servers).

On the Client host, the RMIClient looks up the Middleware's IResourceManager and provides a command-line interface. When a user enters a command, the Client calls the stub, which sends the request to the Middleware.

The Middleware has two roles:

1. Registering its own IResourceManager so that Clients can locate it.

2. Connecting to the Flight, Car, and Room registries and forwarding client requests to the appropriate Resource Manager.

Each Resource Manager handles domain-specific actions through an RMIResourceManager. For example, a Flight request passes from the Client to the Middleware, then to the Flight server's skeleton, which unmarshals the message, executes it, and returns the result via the Middleware.

Customer handling is done through replication across all Resource Managers. For instance, an AddCustomerID request is routed to all three Resource Managers, ensuring future requests involving that Customer can be handled locally. While this may cause scalability issues as customers grow and the Resource Managers become bloated, we believe it was sufficient for this project.

For bundle operations, the Middleware first validates availability of cars and rooms, then checks flight seats. Reservations succeed only if all requested resources are available, ensuring that Bundle remains atomic (either all resources are booked, or none are).

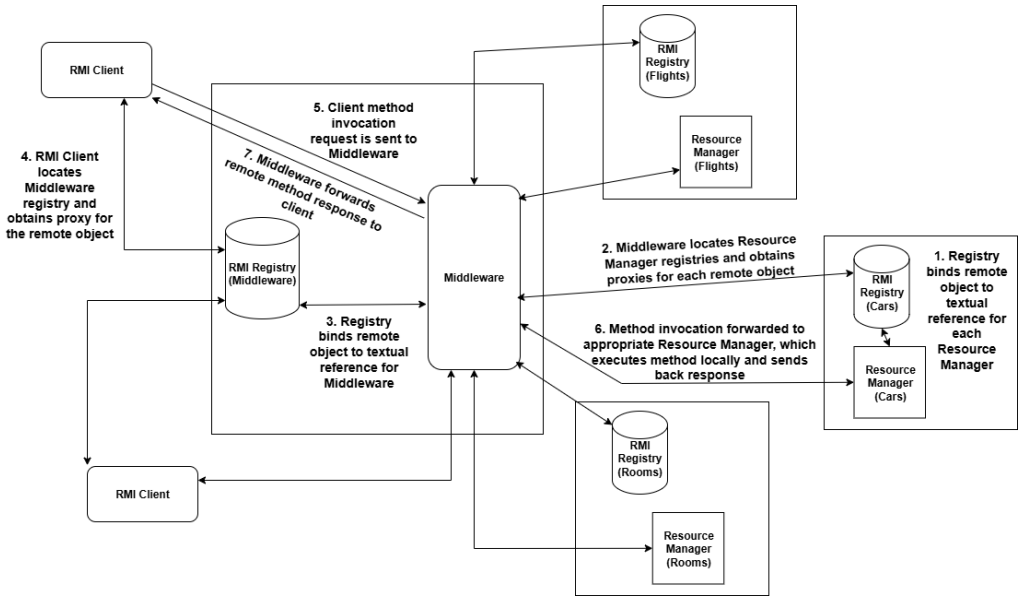


Figure 1: System diagram for RMI architecture

2.2 TCP

As a second task in this project, we implemented the same distributed system from before using TCP for P2P communication. When started, the TCP Resource Manager creates a ServerSocket and listens on a specified port for a single TCP connection from the TCP Middleware. Once a connection is established with the Middleware, input and output streams are created between each Resource Manager and the Middleware. When started, as mentioned above, the Middleware establishes a TCP connection between each Resource Manager using TCP Sockets. It also creates input and output streams to send and receive messages. The Middleware then creates its own ServerSocket, allowing Clients to establish a connection directly with the Middleware.

To establish a connection, the Client sends a TCP connection request to the Middleware, which acknowledges this connection request. Once established, the Client creates input and output streams for message transmission. Note that when a connection is established, the Middleware spawns a new ClientHandler thread that manages the connection between the Middleware and Client. This is done to avoid the Middleware from blocking on Client requests, as each thread maintains its own connection and input/output streams. This also avoids the potential concurrency issue of message corruption that could occur if multiple Clients write (asynchronously) to the same stream at the same time. At this point, the Client may invoke commands. When a Client invokes a command, the message is passed into its output stream to the Middleware.

In this implementation, messages are created using TCP Message objects (defined in the TCPMessage.java class which implements the Java Serializable interface). These objects are used for message passing throughout the TCP Client, Middleware and Resource Managers. They contain useful information about each message, such as the command name and parameters for requests, and response values (integer, boolean, String) for responses. Once this message reaches the Middleware, it routes the message to the appropriate Resource Manager based on the method name indicated inside of the message. The Resource Manager receives this message (as a TCPMessage object) in its input stream, and unpacks the message details (method name, parameters) to perform the correct operation. When this operation is completed, the Resource Manager appends a new message (again as a TCPMessage object) containing a response value to its output stream. The Middleware then receives

this response in its input stream and sends it to the Client, who receives the message from its input stream and reads the response value.

For Customer handling, the Middleware keeps all Customer objects and their reservations, including Flights, Cars, and Rooms. Each Resource Manager only stores a small reference to the Customer to verify requests. When a new customer is added, the Middleware creates a Customer object locally and tells all Resource Managers to register the same customer. Queries and billing are handled directly by the Middleware using the Customer objects. For reservations, the Middleware first checks that the customer exists, forwards the request to the appropriate Resource Manager, and if it succeeds, updates the customer's record with the new booking.

For bundle operations, the Middleware first checks that the customer exists and verifies availability of all requested resources, including flights, cars, and rooms. It attempts to reserve each resource one by one, but only finalizes the reservations if all of them succeed. If any resource cannot be booked, none of the reservations are applied, ensuring the operation is atomic. Once successful, the Middleware updates the customer's record with all the new bookings.

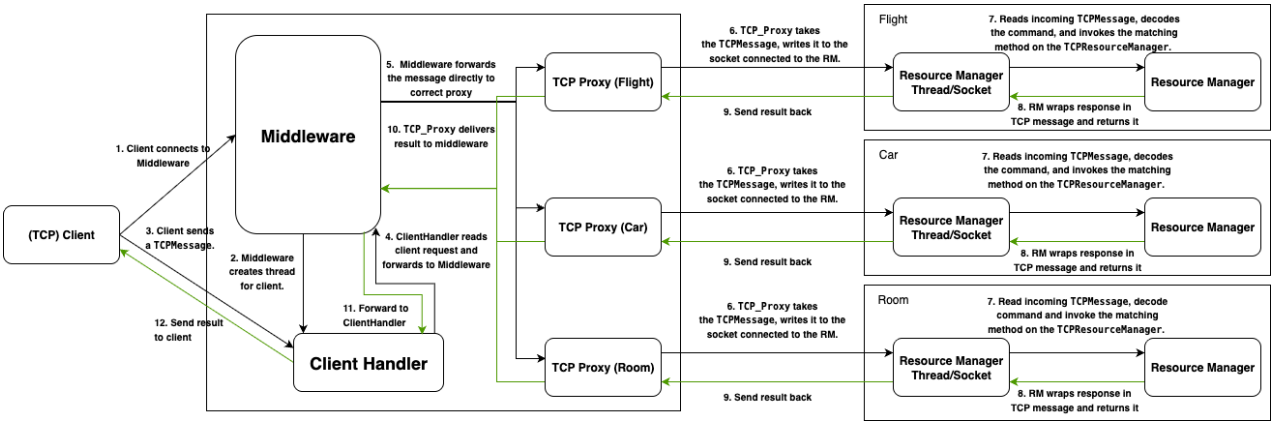


Figure 2: System diagram for TCP architecture

3 Testing

Below lists a series of tests that were conducted to verify the correctness of our implementations. We conducted several other tests to check for edge cases, including (but not limited to): reserving a Bundle with either Flights, Cars or Rooms unavailable, deleting Cars, Rooms or Flights that were booked by Customers and querying previously deleted Cars, Rooms or Flights.

- **Command Sequence:**

```
AddFlight, 20, 1, 300
AddCars, Rome, 0, 150
AddRooms, Rome, 2, 100
Bundle, 1, 20, Rome, 1, 1
```

  - **Description:** A bundle request is made with one available flight, one unavailable car, and one available room.
  - **Expected Result:** The bundle should fail, since not all requested resources are available.
  - **Observed Result:** The bundle reservation was rejected; no resources were booked.

- **Command Sequence:**

```
AddFlight, 50, 2, 400
AddCars, Berlin, 2, 250
AddRooms, Berlin, 2, 180
Bundle, 2, 50, Berlin, 1, 1
```

  - **Description:** A bundle request is made with one flight, one car, and one room, all of which are available.
  - **Expected Result:** The bundle should succeed, with all resources reserved together.
  - **Observed Result:** The bundle reservation was successful; all resources were booked atomically.

- **Command Sequence:**

```
AddFlight, 30, 1, 200
AddCars, Paris, 1, 100
AddRooms, Paris, 1, 80
ReserveFlight, 1, 30
ReserveCar, 1, Paris
ReserveRoom, 1, Paris
DeleteFlight, 30
DeleteCars, Paris
DeleteRooms, Paris
```

  - **Description:** A customer reserves one flight, one car, and one room. The same resources are then deleted from the system.
  - **Expected Result:** Deletions should fail, since the resources are still associated with an active customer.
  - **Observed Result:** The system prevented deletion of the resources until the associated customer was removed.

4 Collaboration

For this assignment, we met on three separate occasions on the third floor of the Trottier building, with meeting dates and times as follows: September 20th 1pm-4:15pm, September 23rd 1pm-3:30pm, and September 26th 3-5:30pm. We used WhatsApp to communicate and update each other regarding task progress. We discussed and agreed on the architecture of both RMI and TCP together. For the RMI implementation, Ion was responsible for implementing the singular methods for the RMI Middleware (e.g. QueryFlight, AddCars, etc.) while Karl was responsible for implementing the Bundle method. We both equally collaborated on the main function of the RMI Middleware class which consists of locating RMI registries and proxy creation. For the TCP implementation, Ion was responsible for implementing the TCP ResourceManager, while Karl was mainly responsible for developing the TCP Client. Both of us contributed to developing the TCP Middleware. Ion mainly worked on connection handling between the Resource Managers and Middleware as well as request and response routing between the Client and Resource Managers, while Karl was responsible for concurrent Client connection handling and Customer handling.